

Guide d'intégration API pour les acteurs partenaires

Référence complète des API exposées par la plateforme TrustWedge pour une intégration dans le système d'information des institutions partenaires : entrées attendues, sorties produites, authentification et droits d'accès, pour chaque point d'intégration.

Université

Entreprise

Banque

Notaire

Tiers vérificateur

Version du document : 1.0 — 18 septembre 2026

Destiné aux équipes techniques des institutions partenaires intégrant l'API TrustWedge à leur SI

Toute la documentation projet : voir docs/README.md

Sommaire

1. Conventions communes

- URL de base et environnements
- Authentification (JWT Bearer)
- Format des requêtes et réponses
- Gestion des erreurs
- Rôles applicatifs

2. Authentification — création et connexion de compte

3. Vérification publique de documents (sans authentification)

4. Divulcation sélective et partage

5. Émission de documents (émetteurs)

6. Gestion des comptes rattachés

7. Workflows métier (vérification, demande de prêt)

8. Transfert de titre foncier à triple signature

9. Vérification d'identité citoyenne (KYC)

10. Notifications

11. Tableau récapitulatif de tous les endpoints

12. Hors périmètre — administration de la plateforme

1. Conventions communes

1.1 URL de base et environnements

Toutes les API décrites dans ce document transitent par un point d'entrée HTTP unique (passerelle Kong) et partagent le préfixe `/api` :

Environnement	URL de base
Démonstration locale	<code>http://localhost:8000/api</code>
Production / pilote	<code>https://<domaine-ou-ip-de-la-plateforme>:8443/api</code>

Un chemin comme `/documents/verify/{token_id}` dans ce document doit donc être appelé à l'URL complète `{URL de base}/documents/verify/{token_id}`.

1.2 Authentification (JWT Bearer)

La quasi-totalité des endpoints (à l'exception de ceux explicitement marqués **Non**) exigent un jeton JWT obtenu via `POST /api/auth/login` (voir §2), transmis dans l'en-tête HTTP suivant :

```
Authorization: Bearer <access_token>
```

Le jeton encode l'identifiant et le rôle applicatif de l'appelant (`sub` , `role`) et expire après `JWT_EXPIRATION` heures (variable d'environnement propre à chaque déploiement — se renseigner auprès de l'exploitant de la plateforme). Un jeton expiré ou invalide produit une réponse `401 Unauthorized`.

1.3 Format des requêtes et réponses

Élément	Convention
Corps de requête	<code>Content-Type: application/json</code> , sauf mention contraire (upload de fichier en <code>multipart/form-data</code>)
Corps de réponse	<code>application/json</code> dans tous les cas, y compris les erreurs
Dates	ISO-8601 avec fuseau (ex. <code>2026-09-18T10:32:00+00:00</code>)
Identifiants de document	<code>doc_type</code> + <code>doc_key</code> : référence métier à retenir côté intégrateur (ex. n° de diplôme, de titre foncier) — voir <code>GET /documents/by-reference</code> . <code>token_id</code> : entier, identifiant du NFT ERC-721 on-chain, détail d'implémentation interne utile pour enchaîner sur <code>/versions</code> ou <code>/owner-at</code>
Adresses	Adresses Ethereum au format hexadécimal standard (<code>0x...</code>)

1.4 Gestion des erreurs

Toute erreur renvoie un code HTTP standard accompagné d'un corps JSON de la forme `{"detail": "message explicite"}`. Codes rencontrés dans ce document :

Code	Signification
400	Requête invalide (champ manquant, valeur incohérente, règle métier non respectée)
401	Authentification absente, invalide ou expirée
403	Authentifié mais rôle/droit insuffisant pour cette action
404	Ressource introuvable (document, workflow, compte...)
409	Conflit avec l'état actuel de la ressource (ex. document déjà transféré)
410	Ressource expirée ou révoquée (lien de partage, preuve de divulgation)

1.5 Rôles applicatifs

Chaque compte TrustWedge porte un rôle unique, qui détermine les endpoints accessibles :

Rôle	Type d'acteur
ISSUER	Émetteur de documents (typiquement une université)
VERIFIER	Vérificateur/émetteur secondaire (typiquement une entreprise)
BANK	Banque — instruction de prêts, revue KYC
NOTARY	Notaire — validation des transferts de titres, revue KYC
USER	Citoyen, titulaire de documents
ADMIN	Administrateur de la plateforme (hors périmètre de ce guide — voir §12)

2. Authentification

Module `auth` — unique émetteur de jetons JWT sur la plateforme.

POST `/auth/register`

Authentification requise : **Non** · **Qui :** tout acteur créant un compte autonome (typiquement un citoyen)

Crée un compte applicatif avec mot de passe. Pour un compte rattaché à une institution (étudiant, employé, client), préférer `POST /actors/create-managed-user` (§6.1), qui génère en plus un DID et une pièce d'identité numérique.

ENTRÉE

Champ	Type	Description
<code>email</code>	string	Identifiant de connexion, unique
<code>password</code>	string	Mot de passe en clair (haché côté serveur, bcrypt)
<code>full_name</code>	string	Nom affiché
<code>role</code>	string	Rôle applicatif (défaut <code>USER</code>)
<code>address</code>	string	Adresse Ethereum associée au compte

SORTIE — 200 OK

```
{
  "id": 42,
  "did": "did:ethr:0xabc...",
  "address": "0xabc...",
  "email": "contact@partenaire.sn",
  "full_name": "Mariama Ndiaye",
  "role": "USER",
  "is_active": true,
  "created_at": "2026-09-18T10:00:00+00:00"
}
```

Erreurs : **400** email déjà enregistré

POST /auth/login

Authentification requise : Non · **Qui :** tout compte existant

Vérifie le mot de passe (bcrypt) et émet le jeton JWT à utiliser sur tous les autres endpoints (§1.2). Un compte provisionné par un administrateur sans mot de passe initial est refusé ici tant qu'un mot de passe ne lui a pas été explicitement défini côté plateforme.

ENTRÉE

Champ	Type	Description
email	string	
password	string	

SORTIE — 200 OK

```
{
  "access_token": "eyJhbGciOiJIUzI1NiIs... ",
  "token_type": "bearer",
  "user": { "id": 42, "did":
    "did:ethr:0xabc...",
    "email": "contact@partenaire.sn",
    "full_name": "...",
    "role": "ISSUER", "is_active": true,
    "created_at": "2026-09-18T10:00:00+00:00" }
}
```

Erreurs : **401** identifiants invalides ou compte sans mot de passe défini

3. Vérification publique de documents

Module `verify` — **point d'intégration principal pour un tiers** (recruteur, autre banque, administration...) : lecture directe de la blockchain, sans clé de signature, sans dépendance à la disponibilité de l'émetteur d'origine. La plupart de ces endpoints sont **publics**, pensés pour être appelés directement depuis le SI d'un tiers (ex. un ATS vérifiant automatiquement un diplôme à la réception d'une candidature).

GET `/documents/verify/{token_id}`

Authentication requise : **Non** · **Qui** : n'importe quel tiers

Vérification canonique d'un document à partir de son identifiant on-chain. Lecture blockchain uniquement (`eth_call`) — ne peut jamais renvoyer un état falsifié par la plateforme elle-même.

ENTRÉE

Paramètre	Type	Description
<code>token_id</code>	int (chemin)	Identifiant du document

SORTIE — 200 OK

```
{
  "isValid": true,
  "issuerDid": "did:ethr:0xissuer...",
  "issuer": "0xissuer...",
  "owner": "0xowner...",
  "docType": "DIPLOMA",
  "docKey": "UCAD-2026-LMD-0421",
  "ipfsCid": "bafybeigd...",
  "owner_name": "Mariama Ndiaye",
  "issuer_name": "Université Cheikh Anta Diop"
}
```

`isValid: false` si le document a été révoqué — ce n'est pas une erreur HTTP, le champ doit être testé côté appelant.

GET /documents/by-reference

Authentification requise : Non · **Qui :** n'importe quel tiers

Méthode recommandée quand l'appelant connaît sa référence métier (n° de diplôme, de titre foncier, de permis, ...) mais pas le `token_id` interne TrustWedge — cas courant pour un tiers externe. Même réponse que `/documents/verify/{token_id}`, avec le `token_id` en plus pour enchaîner sur `/versions` ou `/owner-at` sans refaire une recherche par référence.

ENTRÉE

Paramètre	Type	Description
<code>doc_type</code>	string (query)	Type de document, ex: <code>DIPLOMA</code>
<code>doc_key</code>	string (query)	Référence métier, ex: <code>UCAD-2026-LMD-0421</code>

SORTIE — 200 OK

```
{
  "isValid": true,
  "issuerDid": "did:ethr:0xissuer...",
  "issuer": "0xissuer...",
  "owner": "0xowner...",
  "docType": "DIPLOMA",
  "docKey": "UCAD-2026-LMD-0421",
  "ipfsCid": "bafybeigd...",
  "owner_name": "Mariama Ndiaye",
  "issuer_name": "Université Cheikh Anta Diop",
  "token_id": 42
}
```

Erreurs : **404** si aucun document ne correspond à ce couple `doc_type` / `doc_key`

POST /documents/verify-file

Authentification requise : Oui · **Qui :** tout compte authentifié

Identifie un document à partir du seul fichier déposé (sans connaître son `token_id`) : calcule le CID du fichier et le recherche dans le registre. Requête `multipart/form-data` (champ `file`).

ENTRÉE

Champ	Type	Description
<code>file</code>	fichier	Le document à identifier

SORTIE — 200 OK

```
{ "isValid": true, "docType": "DIPLOMA",
  "docKey": "...",
  "owner_name": "...", "issuer_name": "...",
  "token_id": 42, "computed_cid": "bafybei...",
  "match": true }
```

Erreurs : **404** ce fichier ne correspond à aucun document connu

POST /documents/{token_id}/verify-file

Authentification requise : Oui · **Qui :** tout compte authentifié

Variante ciblée : détecte une falsification en comparant le CID d'un fichier fourni à celui enregistré on-chain pour un `token_id` déjà connu.

ENTRÉE

Paramètre	Type	Description
<code>token_id</code>	int (chemin)	
<code>file</code>	fichier (multipart)	

SORTIE — 200 OK

```
{ "match": true, "computed_cid": "bafybei...",  
  "onchain_cid": "bafybei..." }
```

GET /documents/{token_id}/versions

Authentification requise : Oui · **Qui :** tout compte authentifié

Historique complet des versions d'un document (utile pour un titre foncier ayant changé de propriétaire) — lu directement on-chain.

ENTRÉE

Paramètre	Type
<code>token_id</code>	int (chemin)

SORTIE — 200 OK

```
{ "token_id": 42, "doc_type": "LAND_TITLE",  
  "doc_key": "TF-2024-0001",  
  "current_owner": "0x...",  
  "current_owner_name": "Mariama Ndiaye",  
  "current_owner_did": "did:ethr:0x...",  
  "is_active": true,  
  "is_transferable": true, "attributes": [ {  
    "key": "location",  
    "value": "Dakar, Almadies", "valueType":  
    "string" } ],  
  "versions": [ { "version_index": 0, "cid":  
    "bafy...", "owner": "0x...",  
    "owner_name": "...", "timestamp":  
    1737000000, "is_current": false },  
    { "version_index": 1, "cid": "bafy...",  
    "owner": "0x...",  
    "owner_name": "...", "timestamp":  
    1758000000, "is_current": true } ] }
```

GET /documents/{token_id}/owner-at

Authentification requise : **Oui** · **Qui :** tout compte authentifié

Retrouve le propriétaire d'un document à un instant donné (utile pour reconstituer une chaîne de propriété à une date de référence).

ENTRÉE

Paramètre	Type
token_id	int (chemin)
timestamp	int (requête) — Unix, secondes

SORTIE — 200 OK

```
{ "token_id": 42, "timestamp": 1737000000,
  "owner": "0xowner..." }
```

4. Divulcation sélective et partage

Permet à un citoyen de ne divulguer qu'un sous-ensemble des champs d'un document (ex. son nom et son statut de diplômé, sans le détail de ses notes), ou de partager un document entier via un lien à durée de vie limitée. Module `exchange`.

POST `/documents/verify-shared`

Authentification requise : **Non** · **Qui :** n'importe quel tiers scannant un QR de divulgation sélective

Vérifie une preuve de divulgation sélective générée par le wallet mobile du citoyen (signature ECDSA d'un sous-ensemble de champs). Trois garanties vérifiées dans l'ordre : fraîcheur (preuve valable 5 minutes), authenticité (signature), intégrité (chaque champ divulgué est recomparé à la valeur on-chain).

ENTRÉE

Champ	Type	Description
<code>token_id</code>	int	
<code>owner</code>	string	Adresse du titulaire déclaré
<code>fields</code>	[[key,label,value]]	Champs divulgués
<code>timestamp</code>	int	Epoch ms, au moment de la signature
<code>signature</code>	string	Signature EIP-191 du propriétaire

SORTIE — 200 OK

```
{ "valid": true, "doc_type": "DIPLOMA",  
  "doc_key": "UCAD-2026-...",  
  "owner_name": "Mariama Ndiaye",  
  "fields": [ { "key": "mention", "label":  
    "Mention", "value": "Bien" } ],  
  "verified_at": "2026-09-18T10:05:00+00:00" }
```

Erreurs : **400** signature illisible/horodatage invalide · **409** document révoqué, propriétaire ne correspond plus, ou champ falsifié · **410** preuve expirée (>5 min)

POST /shares

Authentification requise : Oui · **Qui :** le titulaire du document

Génère un lien de partage à durée de vie limitée pour un document possédé. Le lien ne contient aucune donnée : son ouverture relit toujours l'état réel on-chain.

ENTRÉE

Champ	Type	Description
token_id	int	
access_level	string	"view" ou "download", défaut view
expires_in_days	int, optionnel	Défaut 7 jours ; null = sans expiration

SORTIE — 200 OK

```
{ "id": 7, "share_token": "kR3f...", "token_id": 42,
  "doc_type": "DIPLOMA", "doc_key": "UCAD-2026-...",
  "access_level": "view", "expires_at": "2026-09-25T10:00:00+00:00",
  "revoked": false, "created_at": "2026-09-18T10:00:00+00:00" }
```

Erreurs : 403 non propriétaire du document · 404 document introuvable

GET /shares/mine

Authentification requise : Oui · **Qui :** tout compte authentifié

Liste des liens de partage créés par l'appelant.

ENTRÉE

Aucun paramètre.

SORTIE — 200 OK

```
[ { "id": 7, "share_token": "kR3f...",
  "token_id": 42, ... } ]
```

GET /shares/{share_token}

Authentification requise : **Non** · **Qui :** quiconque détient le lien

Résolution publique d'un lien de partage — c'est la raison d'être du lien. Relit toujours l'état réel on-chain (jamais de cache), un document révoqué depuis l'émission du lien n'apparaît donc jamais comme valide.

ENTRÉE

Paramètre	Type
share_token	string (chemin)

SORTIE — 200 OK

```
{ "isValid": true, "docType": "DIPLOMA",
  "docKey": "UCAD-2026-...",
  "owner_name": "Mariama Ndiaye", "issuer_name":
  "UCAD",
  "access_level": "view", "can_download": false
}
```

Erreurs : **404** lien introuvable · **410** lien révoqué ou expiré

DELETE /shares/{share_token}

Authentification requise : **Oui** · **Qui :** le créateur du lien

Révoque un lien de partage — il devient immédiatement inutilisable.

ENTRÉE

Paramètre	Type
share_token	string (chemin)

SORTIE — 200 OK

```
{ "revoked": true }
```

Erreurs : **403** pas à l'origine du lien · **404** lien introuvable

5. Émission de documents

Module `documents`. Réservé aux comptes institutionnels (`ISSUER`, `VERIFIER`, `NOTARY`, `ADMIN`) — c'est le point d'intégration attendu pour qu'une université ou une entreprise émette automatiquement un diplôme ou une attestation depuis son propre système (scolarité, SIRH...).

POST `/documents/issue`

Authentification requise : Oui · Qui : `ISSUER` `VERIFIER` `NOTARY` `ADMIN`

Émet un nouveau document : mint du NFT on-chain, génération automatique du PDF représentatif, upload IPFS, indexation applicative. Le couple (`doc_type`, `doc_key`) doit être unique sur toute la plateforme ; un contenu identique ne peut pas non plus être réémis sous une autre référence.

Le champ `file_content` est obligatoire dans le schéma mais actuellement **ignoré côté serveur** : le PDF est toujours régénéré automatiquement à partir des `attributes` fournis. Envoyer n'importe quelle valeur non vide (ex. `"placeholder"`) pour satisfaire la validation.

ENTRÉE

SORTIE — 200 OK

Champ	Type	Description
<code>owner_did</code>	string	DID du titulaire
<code>doc_type</code>	string	Ex. <code>DIPLOMA</code> , <code>LAND_TITLE</code>
<code>doc_key</code>	string	Référence maître <code>doc_type</code>
<code>attributes</code>	[[key,value,valueType]]	<code>valueType</code> ∈ <code>string/uint256/address/date/boolean</code>
<code>file_content</code>	string	Ignoré côté serveur (voir note ci-dessus)
<code>filename</code>	string	Nom de fichier indicatif
<code>is_transferable</code>	bool	Défaut <code>false</code> ; <code>true</code> pour un titre foncier

```
{ "id": 118, "token_id": 118,
  "issuer": "0xissuer...", "owner":
  "0xowner...",
  "doc_type": "DIPLOMA", "doc_key": "UCAD-2026-
  LMD-0421",
  "ipfs_cid": "bafybeigd...", "is_active": true,
  "is_transferable": false,
  "attributes": [ { "key": "mention", "value":
  "Bien", "valueType": "string" } ],
  "created_at": "2026-09-18T10:00:00+00:00" }
```

Erreurs : **400** référence déjà utilisée (avec un contenu identique ou différent) · **403** rôle non autorisé à émettre

GET /documents/issued

Authentification requise : **Oui** · **Qui :** tout émetteur (`ADMIN` / `NOTARY` voient tout le registre, les autres uniquement leurs propres émissions)

Liste des documents émis, du point de vue de l'émetteur (utile pour un tableau de bord "diplômes délivrés cette année" côté SI de l'université).

ENTRÉE

Paramètre	Type
<code>doc_type</code>	string, optionnel (requête)

SORTIE — 200 OK (JUSQU'À 50 RÉSULTATS)

```
[ { "token_id": 118, "doc_type": "DIPLOMA",
  "doc_key": "UCAD-2026-...",
  "issuer": "0x...", "owner": "0x...",
  "is_active": true,
  "is_transferable": false, "created_at":
  "2026-09-18T10:00:00+00:00" } ]
```

GET /documents/my

Authentification requise : **Oui** · **Qui :** tout compte authentifié

Liste des documents dont l'appelant est **titulaire** (point de vue du citoyen, ou d'une institution détenant elle-même un document).

ENTRÉE

Aucun paramètre.

SORTIE — 200 OK

```
[ { "id": 3, "token_id": 118, "issuer": "0x...",
  "issuer_address": "0x...",
  "owner": "0x...", "doc_type": "DIPLOMA",
  "doc_key": "UCAD-2026-...",
  "ipfs_cid": "bafy...", "is_active": true,
  "is_transferable": false,
  "attributes": [...], "created_at": "2026-09-
  18T10:00:00+00:00" } ]
```

GET /documents/download/{cid}

Authentification requise : **Oui** · **Qui :** tout compte authentifié

Télécharge le fichier PDF associé à un CID IPFS.

ENTRÉE

Paramètre	Type
<code>cid</code>	string (chemin)

SORTIE — 200 OK

Corps binaire, `Content-Type: application/pdf`.

6. Gestion des comptes rattachés

Module `identity`. Permet à une institution de créer, depuis son propre SI, les comptes de ses étudiants/employés/clients, avec génération automatique de leur identité numérique (DID) et de leur pièce d'identité (`ID_CARD`).

POST /actors/create-managed-user**Authentification requise :** Oui · **Qui :** ISSUER VERIFIER BANK

Crée un compte géré par l'institution appelante et génère son DID. Si la personne existe déjà sur la plateforme (email ou n° de CNI déjà connu), aucun second DID n'est créé : elle est simplement rattachée à l'institution appelante (`already_existed: true` , sans clé privée redonnée).

Rôle appelant	Rôles cible autorisés
ISSUER	USER (étudiant)
VERIFIER	USER (employé) ou VERIFIER (autre représentant de l'entreprise)
BANK	USER (client)

ENTRÉE

Champ	Type	Description
email	string	
full_name	string	
first_name / last_name	string, optionnels	Alimentent la pièce d'identité générée
date_of_birth	string, requis si rôle cible USER	
place_of_birth	string, requis si rôle cible USER	
national_id_number	string, requis si rôle cible USER	N° de CNI — jamais stocké en clair (seul un hash SHA-256 l'est, à des fins de dédoublonnage)
role	string	Défaut USER — voir table ci- dessus

SORTIE — 200 OK

```
{ "id": 57, "did": "did:ethr:0xnew...",  
  "address": "0xnew...",  
  "private_key": "0xabc123...",  
  "public_key": "0x04...", "email":  
  "etudiant@ucad.sn",  
  "full_name": "Ibrahima Fall", "date_of_birth":  
  "2001-03-14",  
  "place_of_birth": "Thiès",  
  "national_id_number": "1234567890123",  
  "role": "USER", "created_at": "2026-09-  
  18T10:00:00+00:00" }
```

⚠ `private_key` n'est retournée qu'une seule fois, à la création — à transmettre immédiatement et de façon sécurisée à la personne concernée, jamais journalisée en clair.

Erreurs : 403 rôle appelant non autorisé à créer des comptes, ou rôle cible non autorisé pour ce type d'institution

GET /did/my

Authentification requise : Oui · **Qui :** tout compte authentifié

Récupère ses propres identifiants (DID, adresse, clé privée déchiffrée à la volée). Utile pour un service technique ayant besoin de signer localement.

ENTRÉE

Aucun paramètre.

SORTIE — 200 OK

```
{ "id": 57, "did": "did:ethr:0x...", "address":  
"0x...",  
  "private_key": "0xabc123...", "public_key":  
"0x04...",  
  "email": "etudiant@ucad.sn", "full_name":  
"Ibrahima Fall", "role": "USER" }
```

POST /did/regenerate

Authentification requise : Oui · **Qui :** tout compte authentifié

Régénère une nouvelle paire de clés (ex. en cas de compromission suspectée). L'ancienne clé est révoquée ; les rôles on-chain sont automatiquement déplacés vers la nouvelle adresse.

ENTRÉE

Aucun paramètre.

SORTIE — 200 OK

```
{ "status": "regenerated", "new_address":  
"0xnew...",  
  "new_did": "did:ethr:0xnew..." }
```

7. Workflows métier

Module `documents`. Machine à états générique utilisée pour la vérification de diplôme, la vérification d'attestation d'emploi et l'instruction d'une demande de prêt. Cycle : `PENDING` → `IN_PROGRESS` → `AWAITING_VERIFICATION` → `IN_PROGRESS` → `AWAITING_NOTARY` → `COMPLETED` (ou `REJECTED` / `CANCELLED` à toute étape autorisée). Le transfert de titre foncier suit un circuit séparé, plus fort cryptographiquement (§8).

POST `/workflows/create`

Authentification requise : Oui · **Qui** : tout compte authentifié (l'appelant devient l'initiateur)

Démarre un nouveau workflow.

ENTRÉE

SORTIE — 200 OK

Champ	Type	Description	{ "workflow_id": 91, "status": "PENDING" }
<code>workflow_type</code>	string	<code>DIPLOMA_VERIFICATION</code> / <code>EMPLOYMENT_VERIFICATION</code> / <code>LOAN_APPLICATION</code> / <code>ID_CARD_ISSUANCE</code> (le transfert foncier utilise §8, pas ce endpoint)	
<code>target_user_id</code>	int, optionnel	Vérificateur ou banque désignée	
<code>document_token_id</code>	int, optionnel	Document concerné	
<code>document_cid</code>	string, optionnel		
<code>workflow_data</code>	objet libre, optionnel	Données métier additionnelles	

7.2 Actions sur un workflow existant

Chaque action renvoie la même forme de réponse : `{ "workflow_id": <int>, "status": <nouvel état> }`. Toutes exigent une authentification, et seule la partie autorisée pour l'étape peut agir (initiateur, vérificateur désigné ou notaire désigné selon l'action).

Méthode & chemin	Qui	Entrée (corps JSON)	Effet
<code>POST /workflows/{id}/start</code>	Initiateur	—	<code>PENDING</code> → <code>IN_PROGRESS</code>

POST /workflows/{id}/request-verification	Initiateur	{ verifier_id, verification_data? }	→ AWAITING_VERIFICATION
POST /workflows/{id}/submit-verification	Vérificateur désigné	{ is_valid, notes? }	→ IN_PROGRESS ou REJECTED
POST /workflows/{id}/request-notary	Initiateur	{ notary_id }	→ AWAITING_NOTARY
POST /workflows/{id}/validate-by-notary	Notaire désigné	{ is_valid, notes?, transaction_hash? }	→ COMPLETED ou REJECTED
POST /workflows/{id}/cancel	Initiateur	{ reason? }	→ CANCELLED (impossible si déjà COMPLETED / REJECTED)

GET /workflows/{workflow_id}/status

Authentication requise : **Oui**

État détaillé d'un workflow, avec son historique d'étapes.

SORTIE — 200 OK

```
{ "workflow_id": 91, "status": "AWAITING_NOTARY",
  "workflow_type": "LOAN_APPLICATION", "initiator_id": 12,
  "target_user_id": 3, "notary_id": null, "document_token_id": 118,
  "workflow_data": {}, "created_at": "...", "updated_at": "...",
  "completed_at": null,
  "steps": [ { "step_name": "start", "action": "start_workflow",
    "status": "DONE", "created_at": "..." } ] }
```

GET /workflows/my

Authentication requise : **Oui**

Liste des workflows où l'appelant est partie prenante (initiateur, vérificateur ou notaire).

ENTRÉE

Paramètre	Type
status	string, optionnel (requête)

SORTIE — 200 OK

```
[ { "id": 91, "type": "LOAN_APPLICATION",
  "status": "AWAITING_NOTARY",
  "initiator_id": 12, "target_user_id": 3,
  "notary_id": null,
  "document_token_id": 118, "workflow_data":
  {},
  "created_at": "...", "updated_at": "..." } ]
```

8. Transfert de titre foncier à triple signature

Module `exchange`. Circuit dédié aux titres fonciers (`is_transferable: true`), plus fort qu'un simple changement de statut applicatif : vendeur et acheteur signent chacun avec leur clé privée, le notaire finalise et exécute le transfert ERC-721 on-chain.

POST `/workflows/transfer/initiate`

Authentification requise : **Oui** · **Qui :** le vendeur (titulaire actuel)

Met en vente un titre possédé, signé de la clé privée du vendeur.

ENTRÉE

Champ	Type
<code>token_id</code>	int
<code>buyer_email</code>	string — l'acheteur doit déjà avoir un compte

SORTIE — 200 OK

```
{ "workflow_id": 205, "status":  
  "AWAITING_VERIFICATION", "tx_hash": "0x..." }
```

Erreurs : **403** non propriétaire · **400** document non transférable, ou vente à soi-même · **404** acheteur introuvable

POST `/workflows/transfer/accept`

Authentification requise : **Oui** · **Qui :** l'acheteur désigné

Accepte l'offre et signe à son tour. Un notaire disponible est automatiquement désigné pour la finalisation.

ENTRÉE

Champ	Type
<code>workflow_id</code>	int

SORTIE — 200 OK

```
{ "workflow_id": 205, "status":  
  "AWAITING_NOTARY", "tx_hash": "0x..." }
```

Erreurs : **403** non acheteur désigné · **400** offre non en attente de signature, ou aucun notaire disponible

POST /workflows/transfer/notary-finalize

Authentification requise : Oui · **Qui :** NOTARY désigné pour ce transfert

Finalise : transfert de propriété exécuté on-chain, nouveau titre PDF régénéré au nom de l'acheteur et republié sur IPFS.

ENTRÉE

Champ	Type
workflow_id	int
notes	string, optionnel

SORTIE — 200 OK

```
{ "status": "completed", "new_cid":  
  "bafybei...", "tx_hash": "0x..." }
```

Erreurs : 403 réservé aux notaires / non désigné pour ce transfert · 400 transfert non en attente de finalisation

9. Vérification d'identité citoyenne (KYC)

Parcours transverse (module `backend`) préalable à une demande de prêt ou un transfert foncier : étapes techniques sans état (OTP, OCR, correspondance faciale — publiques, avant qu'un JWT existe), suivies d'un dossier persisté soumis à la revue d'une banque ou d'un notaire.

9.1 Étapes techniques (public, sans authentification)

Méthode & chemin	Entrée	Sortie
POST <code>/verification/phone/send</code>	<code>{ phone }</code>	<code>{ token, dev_code }</code> — <code>dev_code</code> uniquement en environnement de démo (SMS non branché) ; en production, seul le SMS porte le code
POST <code>/verification/phone/verify</code>	<code>{ token, code }</code>	<code>{ verified: bool }</code>
POST <code>/verification/email/send</code>	<code>{ email }</code>	<code>{ token }</code>
POST <code>/verification/email/verify</code>	<code>{ token }</code>	<code>{ verified: bool }</code>
POST <code>/verification/id-card/extract</code>	<code>{ image_base64 }</code>	<code>{ nom, prenom, date_naissance, num_cni, nationalite }</code> — OCR réel (Tesseract)
POST <code>/verification/face/compare</code>	<code>{ card_image_base64, selfie_image_base64 }</code>	<code>{ matched, similarity, mocked: true }</code> — correspondance simulée , à ne jamais présenter comme probante en production

POST `/verification/kyc/submit`

Authentification requise : Oui · **Qui** : le citoyen (typiquement depuis le wallet mobile, à l'issue des étapes ci-dessus)

Persiste le dossier KYC complet pour permettre une revue humaine.

ENTRÉE

Champ	Type
<code>full_name</code>	string
<code>id_card_number</code>	string, optionnel
<code>id_card_data</code>	objet — sortie brute de l'OCR
<code>phone_verified</code> / <code>email_verified</code> / <code>face_match_passed</code>	bool

SORTIE — 200 OK

```
{ "id": 14, "user_id": 57, "full_name":  
  "Ibrahima Fall",  
  "id_card_number": "1234567890123",  
  "phone_verified": true,  
  "email_verified": true, "face_match_passed":  
  true,  
  "status": "PENDING", "reviewer_notes": null,  
  "created_at": "2026-09-18T10:10:00+00:00",  
  "reviewed_at": null }
```

GET /verification/kyc/pending

Authentification requise : **Oui** · Qui : **BANK** **NOTARY**

File des dossiers KYC à revoir.

ENTRÉE

Paramètre	Type
status	string, optionnel — défaut PENDING

SORTIE — 200 OK

```
[ { "id": 14, "user_id": 57, "full_name": "...",  
  "status": "PENDING", ... } ]
```

Erreurs : **403** réservé à la banque et au notaire

GET /verification/kyc/mine

Authentification requise : **Oui** · Qui : le citoyen concerné

Statut du dossier KYC de l'appelant (le plus récent).

SORTIE — 200 OK

```
{ "id": 14, "status": "APPROVED", ... } // ou null si aucun dossier
```

POST /verification/kyc/{kyc_id}/review

Authentification requise : **Oui** · Qui : **BANK** **NOTARY**

Décision humaine sur un dossier KYC en attente.

ENTRÉE

Champ	Type
approve	bool
notes	string, optionnel

SORTIE — 200 OK

```
{ "id": 14, "status": "APPROVED",  
  "reviewer_notes": "Pièces conformes",  
  "reviewed_at": "2026-09-18T11:00:00+00:00",  
  ... }
```

Erreurs : **403** réservé à la banque et au notaire · **404** dossier introuvable · **400** dossier déjà traité

10. Notifications

Module `backend` (core). Permet à un SI partenaire de relever les événements concernant ses comptes gérés (document reçu, transfert accepté...) sans avoir à interroger chaque endpoint métier individuellement.

Méthode & chemin	Entrée	Sortie
<code>GET /notifications</code>	—	<code>[{ id, type, message, workflow_id, token_id, read, created_at }]</code>
<code>POST /notifications/{id}/read</code>	—	Marque une notification comme lue
<code>POST /notifications/read-all</code>	—	Marque toutes les notifications comme lues
<code>DELETE /notifications</code>	—	Purge les notifications de l'appelant

Toutes exigent une authentification. Types de notification rencontrés : `document_issued`, `transfer_request`, `transfer_accepted`, `notary_requested`, `transfer_completed`.

10.5 Canal temps réel (optionnel)

Un WebSocket `ws://{hôte}:8000/ws/{user_id}` pousse les mêmes événements en temps réel, pour un SI souhaitant éviter le polling de `GET /notifications`. Format des messages poussés :

```
{ "type": "document_issued", "token_id": 118, "doc_type": "DIPLOMA" }
```

11. Tableau récapitulatif

Méthode	Chemin	Auth	Acteurs	Description
Authentification				
POST	/auth/register	Non	Tous	Créer un compte autonome
POST	/auth/login	Non	Tous	Obtenir un jeton JWT
Vérification publique				
GET	/documents/verify/{token_id}	Non	Tout tiers	Vérifier un document par son ID
GET	/documents/by-reference	Non	Tout tiers	Vérifier un document par sa référence métier (doc_type + doc_key)
POST	/documents/verify-file	Oui	Tous	Identifier un document par son fichier
POST	/documents/{token_id}/verify-file	Oui	Tous	Détecter une falsification de fichier
GET	/documents/{token_id}/versions	Oui	Tous	Historique des versions
GET	/documents/{token_id}/owner-at	Oui	Tous	Propriétaire à une date donnée
Divulgence sélective et partage				
POST	/documents/verify-shared	Non	Tout tiers	Vérifier une preuve de divulgation sélective
POST	/shares	Oui	Titulaire	Créer un lien de partage
GET	/shares/mine	Oui	Tous	Lister ses liens de partage
GET	/shares/{share_token}	Non	Tout tiers	Résoudre un lien de partage
DELETE	/shares/{share_token}	Oui	Créateur du lien	Révoquer un lien de partage
Émission de documents				
POST	/documents/issue	Oui	ISSUER, VERIFIER, NOTARY, ADMIN	Émettre un document
GET	/documents/issued	Oui	Émetteurs	Documents émis par l'appelant
GET	/documents/my	Oui	Tous	Documents détenus par l'appelant
GET	/documents/download/{cid}	Oui	Tous	Télécharger le PDF d'un document
Comptes rattachés				
POST	/actors/create-managed-user	Oui	ISSUER, VERIFIER, BANK	Créer un compte géré (étudiant/employé/client)
GET	/did/my	Oui	Tous	Récupérer ses identifiants DID
POST	/did/regenerate	Oui	Tous	Régénérer sa paire de clés
Workflows métier				

POST	/workflows/create	Oui	Tous	Démarrer un workflow
POST	/workflows/{id}/start	Oui	Initiateur	Démarrer
POST	/workflows/{id}/request-verification	Oui	Initiateur	Demander une vérification
POST	/workflows/{id}/submit-verification	Oui	Vérificateur désigné	Soumettre le résultat
POST	/workflows/{id}/request-notary	Oui	Initiateur	Demander une validation notariale
POST	/workflows/{id}/validate-by-notary	Oui	Notaire désigné	Valider/rejeter
POST	/workflows/{id}/cancel	Oui	Initiateur	Annuler
GET	/workflows/{id}/status	Oui	Parties prenantes	État détaillé
GET	/workflows/my	Oui	Tous	Ses workflows
Transfert de titre foncier				
POST	/workflows/transfer/initiate	Oui	Vendeur	Mettre en vente
POST	/workflows/transfer/accept	Oui	Acheteur	Accepter l'offre
POST	/workflows/transfer/notary-finalize	Oui	NOTARY	Finaliser le transfert
KYC				
POST	/verification/phone/send, /phone/verify	Non	Tous	OTP téléphone
POST	/verification/email/send, /email/verify	Non	Tous	OTP email
POST	/verification/id-card/extract	Non	Tous	OCR carte d'identité
POST	/verification/face/compare	Non	Tous	Correspondance faciale (simulée)
POST	/verification/kyc/submit	Oui	Citoyen	Soumettre le dossier KYC
GET	/verification/kyc/pending	Oui	BANK, NOTARY	Dossiers à revoir
GET	/verification/kyc/mine	Oui	Citoyen	Statut de son dossier
POST	/verification/kyc/{id}/review	Oui	BANK, NOTARY	Décision de revue
Notifications				
GET	/notifications	Oui	Tous	Lister ses notifications
POST	/notifications/{id}/read	Oui	Tous	Marquer comme lue
POST	/notifications/read-all	Oui	Tous	Tout marquer comme lu
DELETE	/notifications	Oui	Tous	Purger ses notifications
WS	/ws/{user_id}	—	Tous	Flux temps réel (optionnel)

12. Hors périmètre de ce document

Les endpoints suivants relèvent de l'**administration de la plateforme TrustWedge elle-même** (rôle `ADMIN`, exploité par l'équipe TrustWedge) et non d'une intégration par le SI d'une institution partenaire — ils ne sont donc pas détaillés ici :

- `POST /admin/actors/create` — provisionnement initial d'un compte institutionnel (fait par TrustWedge à l'onboarding d'un partenaire)
- `POST /admin/actors/{id}/set-password` — définition du mot de passe d'un acteur nouvellement créé
- `POST /admin/land-titles/ocr-extract`, `POST /admin/land-titles/import` — import assisté de titres fonciers existants
- `GET /admin/documents`, `GET /admin/workflows` — vue d'ensemble de tous les documents/workflows de la plateforme
- `GET /stats/overview`, `GET /stats/activity`, `GET /network/status` — indicateurs de supervision de la plateforme

Pour toute question sur l'onboarding d'une nouvelle institution ou l'accès à ces fonctions, contacter l'équipe d'exploitation TrustWedge.